

Présentation SASS



- > **INFRASTRUCTURES** SYSTÈMES & RÉSEAUX
- > **CYBERSÉCURITÉ** INFRASTRUCTURES & APPLICATIONS
- > **DEVOPS / SCRIPTING** & AUTOMATISATION
- > **DEVELOPPEMENT** WEB & MOBILE
- > **TRANSFORMATION NUMERIQUE DES ENTREPRISES**

www.adrar-numerique.com

Présentation de Sass



Sass est un langage dynamique de génération de fichier css.

Pourquoi l'utiliser ?

Il permet de faire beaucoup plus de chose que le css comme des variables, Fonctions mathématiques, Fonctions, Boucles, ect.

Présentation de Live Sass Compiler

Comment le mettre en place ?

Il faut télécharger « Live Sass Compiler », ce dernier va permettre dans notre cas de compiler les fichiers sass en css.



Pour ne pas se tromper de version, il faut vérifier que l'auteur est bien « Glenn Marks » .

Premier projet SASS

On va d'abord créer notre premier projet sass. Il devra contenir :

- Un fichier index.html
- Un fichier style.scss

À partir de ce moment-là, comme à votre habitude, vous devez lier votre fichier HTML et Css.

Vous l'avez peut-être remarqué, nous parlons ici de fichier « css ».

C'est à ce moment que notre compilateur intervient ! Une fois que vous avez édité votre fichier scss, vous devez lancer votre extension Live Sass Compiler qui s'occupera donc de générer un fichier css qui sera interprété par le navigateur.

Premier projet SASS

Avant de commencer, il faut établir le cœur de la syntaxe SASS, à savoir **le Nesting**.

Le principe du Nesting est d'**imbriquer** les sélecteurs CSS les uns dans les autres. C'est-à-dire qu'un sélecteur, ainsi que ses règles CSS, se verra être déclaré entre les accolades d'un premier sélecteur.

Une fois **compilé**, ils formeront une **combinaison de sélecteur**. Cela permet de gagner en clarté grâce à une hiérarchie visuelle similaire à celle du HTML.

En css :

```
div {  
  background-color: royalblue;  
}  
div p {  
  color: white;  
}
```

En sass :

```
div {  
  background-color: royalblue;  
  p {  
    color : white;  
  }  
}
```

Premier projet SASS

Comme vu précédemment, il est possible de faire des sélecteurs imbriqués, ce principe s'applique également aux pseudo-classes:

En css :

```
div {  
  background-color: royalblue;  
}  
div p {  
  color: white;  
}  
div :hover {  
  color: salmon;  
}
```

En sass :

```
div {  
  background-color: royalblue;  
  p {  
    color : white;  
  }  
  :hover {  
    color: salmon;  
  }  
}
```


Les variables SASS

Comme nous l'avons dit dans l'introduction, il est possible de créer des variables.
Elles sont définies ainsi :

```
$fond: royalblue;

div {
  background-color: $fond;
  p{
    color : white;
  }
  :hover{
    color: salmon;
  }
}
```

La création d'une variable se fait en **deux étapes** :

1. En premier, on commence par le symbole « \$ » suivi par un **nom de variable**, ici, c'est le mot « fond ».
2. Nous avons **la valeur** « royalblue » qui est un terme par défaut au même titre que « blue », « red », etc...

Les variables SASS

Pour plus de clarté, on peut également créer un deuxième fichier SCSS qui permettra d'importer les variables dans votre fichier principal par **@import** :

style.scss:

```
@import "../variables.scss";

div {
  background-color: $fond;
  p{
    color : white;
  }
  :hover{
    color: salmon;
  }
}
```

variables.scss:

```
$fond: royalblue;
```


Exercice de variable:

- Créer un nouveau répertoire
- Créer un fichier HTML avec le HTML de base, ajoutez un link pour le fichier Css puis ajoutez une div class « container » contenant : un header qui contient une nav qui elle-même contient un lien « Accueil ».
- Créer ensuite un fichier **SCSS** et sous forme de **nesting**, définissez les variables **\$citron**, **\$lumiereBleu** et **\$cyanSombre**.

Note: n'oubliez pas de cliquer en bas à droite de vscode sur « WatchSASS » pour compiler votre fichier.

Les Mixins

En SCSS, il est possible de stocker du code CSS, un peu comme les variables stockent des valeurs. On appelle cela des **mixins** et elles se présentent ainsi :

style.scss:

```
$cyanSombre: darkcyan;

@mixin cardArticle {
  background-color: $cyanSombre;
  color: #333;
  border: 3px solid #333;
  width: 5vw;
  height: 1vh;
}
```

Ensuite, pour utiliser le mixin, on l'appelle via un **include** :

```
article {
  @include cardArticle;
}
```

Les Mixins

Il est également possible d'ajouter des **paramètres** à un mixin, à l'instar des fonctions en JS.
Voici un exemple :

Déclaration de la Mixin:

```
@mixin bordure($couleur, $largeur: 1px) {  
  border: $largeur solid $couleur;  
  border-radius: 5px;  
}
```

Appelle de la Mixin:

```
.button {  
  @include bordure(blue);  
}  
  
.card {  
  @include bordure(red, 2px);  
}
```

Ici, notre variable a été déclarée dans les parenthèses du mixin, ce qui nous permet de l'utiliser en tant que valeur. Attention, un paramètre n'aura pas la même portée qu'une variable déclarée en dehors de votre mixin !

Les Mixins

L'avantage des Mixins est qu'ils évitent de répéter du code lors de son écriture.

Toutefois, si ils sont appliquées à plusieurs sélecteurs, le code CSS compilé montrera des répétitions de code, chose qu'on souhaite éviter selon les bonnes pratiques du DRY (Don't Repeat Yourself).

Sass

Doc officielle SASS : <https://sass-lang.com/documentation/>

Exercice mixin:

- Créer un nouveau répertoire avec un fichier html et un fichier SCSS puis faire un link CSS. Créez ensuite un mixin nommé « lesDeuxPiedsDansLaMarge » avec des paramètres représentant 10px pour les marges de chaque côté, et appliquez-les à la propriété margin.
- Appliquez ensuite ce mixin à une div possédant une taille et une couleur, en surchargeant de 30 px les marges en haut et à gauche.

Les Extends

Les extensions ou « Extends » de SASS sont similaires aux mixins, mais au lieu de dupliquer un ensemble de propriété CSS, elles dupliquent des **sélecteurs**.

On déclare une extension de la façon suivante : ***%nomExtension {règlesCSS}***.

Tous nos boutons auront ces propriétés

Primary-tim-button aura les propriétés de **.button** et nous surchargerons le background-color en rouge.

```
.button {  
    padding: 10px 20px;  
    background-color: blue;  
    color: white;  
}  
  
.primary-tim-button {  
    @extend .button;  
    background-color: red;  
}
```

Les Extends

L'idéal est de créer des règles via des extends qui ne seront pas interprétés par le navigateur, pour cela, nous rajoutons le symbole "%nomDeMonExtend".
En faisant cela, nous rendons notre css moins lourd et plus facilement maintenable.

style.scss

```
%boutons {
  padding: 10px 20px;
  background-color: blue;
  color: white;
}

.primary-tim-button {
  @extend %boutons;
  background-color: red;
}
```

style.css

```
.primary-tim-button {
  padding: 10px 20px;
  background-color: blue;
  color: white;
}

.primary-tim-button {
  background-color: red;
}
```

Navigateur

```
.primary-tim-button {
  background-color: red;
}

.primary-tim-button {
  padding: 10px 20px;
  background-color: blue;
  color: white;
}
```


Les Extends

Vous avez la possibilité d'organiser vos **@extend** et **mixins** de la même manière que vos variables, en les plaçant dans des fichiers distincts pour améliorer la lisibilité et la réutilisabilité de votre code Sass.

Vous comprenez donc que comme les Mixins, les Extends contribuent fortement à l'écriture d'un code lisible qui suit le principe du Don't Repeat Yourself déjà cité précédemment.